

Quaderno di lavoro

Che cosa è cambiato e perché

Settembre 2026 — dal laboratorio emulato
al datapath reale

Progetto **IPA/eBPF**

inferenza di una rete neurale dentro il kernel, a bordo del pacchetto

Ogni numero di questo quaderno viene da una misura eseguita,
non da una stima. Dove una cosa non è stata verificata, è scritto.

Indice

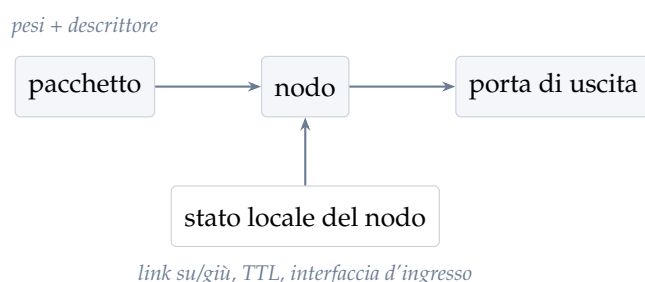
1	Di che cosa stiamo parlando	2
1.1	L'idea in una pagina	2
1.2	Dove gira: XDP	2
1.3	Le tre pipeline	2
2	La matematica che sta dentro il kernel	3
2.1	Perché i numeri sono interi	3
2.2	Il problema dei bias, e l'errore che ci stava dentro	3
2.3	Il TTL: dividere il prodotto, mai il valore	4
3	Le classi non sono porte	5
3.1	Il problema	5
3.2	La soluzione: una catena esplicita	5
3.3	Un errore che questa separazione ha fatto emergere	6
4	Il motore non è lo scenario	7
4.1	Com'era	7
4.2	Com'è adesso: tre strati	7
4.3	La prova	8
5	Dall'emulatore al kernel vero	9
5.1	Che cosa provavano i test di prima	9
5.2	Il banco nuovo	9
5.3	Tre difetti del test, trovati misurando	9
5.4	Il risultato	10
6	La feature che non c'era	11
6.1	Il sintomo	11
6.2	Perché nessun test lo vedeva	11
6.3	La correzione	11
6.4	La prova che adesso è viva	12
6.5	Il prezzo, misurato	12
7	I numeri, e che cosa dicono davvero	13
7.1	La tabella	13
7.2	Primo risultato: la dimensione non predice la velocità	13
7.3	Secondo risultato: quanto valgono le latenze assolute	13
7.4	Terzo risultato: la genericità ha un costo di verifica	14
7.5	Che cosa questi numeri non dicono	14
8	Quel che resta	15
8.1	Fatto	15
8.2	Aperto	15
8.3	Il filo conduttore	15

Di che cosa stiamo parlando

1.1 L'idea in una pagina

Un router normale decide dove mandare un pacchetto consultando una tabella riempita da un protocollo di routing. L'idea di **IPA** (*Intelligent PAckets*) è diversa: il pacchetto **porta con sé** una piccola rete neurale, e ogni nodo che attraversa la esegue per decidere l'uscita.

Niente segnalazione, niente tabella condivisa: la decisione nasce dai dati che il nodo ha sotto mano in quel momento.



Il modello è una rete piccola: quattro tipi di ingresso, due strati nascosti da quattro neuroni, sette uscite. In tutto **319 pesi**. Deve stare in un header ed essere eseguibile in decine di nanosecondi.

1.2 Dove gira: XDP

Il codice gira in **eBPF** agganciato a **XDP**, il punto più precoce in cui il kernel Linux permette di toccare un pacchetto: prima che venga costruita la struttura `sk_buff`, quindi prima di quasi tutto il resto dello stack di rete.

Questo impone un vincolo forte, che spiega metà delle scelte di progetto: un programma eBPF deve essere **dimostrato sicuro** da un componente del kernel chiamato *verificatore* prima di poter girare. Niente cicli di lunghezza ignota, niente accessi non controllati alla memoria, niente ricorsione. Tutto ciò che nel documento sembra contorto — cicli srotolati, tetti dichiarati a priori, tabelle al posto di calcoli — nasce da lì.

1.3 Le tre pipeline

Lo stesso modello è implementato in tre modi, per misurare un compromesso.

	i pesi stanno	cambiare modello
P1 hardcoded	scritti nel codice C	ricompilare
P2 template	in una mappa	scrivere la mappa
P3 modular	in una mappa, uno strato per programma	scrivere la mappa

P1 è veloce perché il compilatore vede i pesi e li semplifica; è rigido perché ogni modello nuovo richiede una ricompilazione. P2 e P3 sono l'opposto. Il lavoro di settembre serve a rendere questo confronto *misurabile davvero*.

La matematica che sta dentro il kernel

2.1 Perché i numeri sono interi

Nel kernel non ci sono numeri in virgola mobile: eBPF lavora con interi. Il modello però è addestrato in virgola mobile. Serve una conversione, e si chiama **quantizzazione**.

Ogni peso reale w diventa un intero a 8 bit:

$$w_{\text{int}} = \text{round}(w \cdot s)$$

dove s è il **fattore di scala**, uno solo per tutta la rete.

Perché il risultato stia in un byte con segno serve

$$s \cdot \max_i |w_i| \leq 127.$$

Sul modello depositato $\max |w| = 5,098$, quindi $s \leq 24,9$. Due scelte convivono nel progetto: $s = 24$ (il massimo intero ammesso) e $s = 16$ (la potenza di due più vicina per difetto, che rende le divisioni degli shift).

Superare quel limite non dà un errore: dà un *overflow silenzioso*. I pesi grandi si troncano e il modello sbaglia senza che nulla lo segnali. Per questo i test distinguono le scale *rappresentabili* da quelle che saturano, e misurano solo sulle prime.

2.2 Il problema dei bias, e l'errore che ci stava dentro

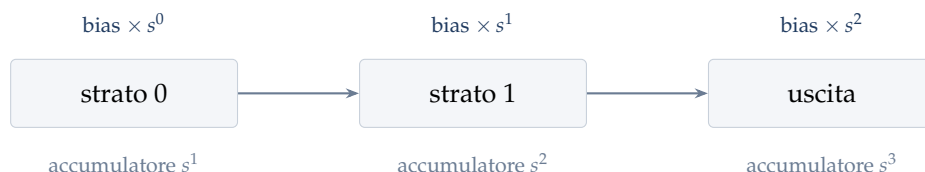
Qui sta l'errore più sottile trovato in settembre, e vale la pena seguirlo passo passo perché è istruttivo.

Quando moltiplichiamo un ingresso per un peso quantizzato, il risultato porta un fattore s in più rispetto al valore reale:

$$x \cdot w_{\text{int}} = x \cdot w \cdot s.$$

Se impiliamo gli strati, quel fattore si accumula. Dopo $\ell + 1$ moltiplicazioni l'accumulatore dello strato ℓ porta $s^{\ell+1}$.

Ma il **bias** è memorizzato come un peso qualsiasi, cioè con un solo fattore s . Sommarlo così com'è significa sommare una quantità sistematicamente troppo piccola. Va riportato nelle unità dell'accumulatore:



La regola è semplice: allo strato ℓ (contando da zero) il bias va moltiplicato per s^ℓ .

L'errore

Il modello di riferimento in Python — quello con cui si controlla che il datapath calcoli la cosa giusta — applicava s^0 e s^1 correttamente ai primi due strati, e lasciava l'**ultimo strato** a s^0 invece di s^2 .

Il bias dell'uscita era sottopesato di $s^2 = 24^2 = 576$.

Perché nessuno se n'era accorto per mesi

Tutti i test eseguivano il modello con **tutti i link attivi**. In quella condizione gli accumulatori sono grandi e un errore di 576 non sposta mai il massimo. Basta spegnere qualche link perché gli accumulatori si riducano e l'errore decida l'esito.

Nello stesso file c'era una seconda funzione di riferimento, scritta per forme arbitrarie, che usava la formula generale s^ℓ ed era **già corretta**. Due riferimenti, uno giusto e uno sbagliato, e quello sbagliato era l'unico mai esercitato con link spenti.

Corretto l'errore, il riferimento e le tre pipeline concordano su tutti i casi provati.

La lezione non è "c'era un bug". È che **un test che varia un solo ingresso non è un test**: verifica una condizione e la ripete. Ci torneremo nel capitolo 5, dove lo stesso schema si ripresenta in una forma ancora più netta.

2.3 Il TTL: dividere il prodotto, mai il valore

Il modello è addestrato con il TTL *normalizzato*, cioè ttl/T con $T = 30$ il valore iniziale. Con i numeri interi

$$\frac{\text{ttl}}{30} = 0 \quad \text{per ogni } \text{ttl} < 30,$$

e la feature sparirebbe. La divisione va quindi applicata al *prodotto*:

$$\left\lfloor \frac{\text{ttl} \cdot w_{\text{int}}}{T} \right\rfloor \quad \text{e non} \quad \left\lfloor \frac{\text{ttl}}{T} \right\rfloor \cdot w_{\text{int}}.$$

Nel C generato si legge esattamente così:

```
(((__s64)_ttl * 11LL) / 30LL)
```

Le classi non sono porte

3.1 Il problema

La rete produce sette numeri; il più grande vince. Ma *che cosa significa* la classe vincente?

All'inizio di settembre la risposta era sparsa e incoerente. In punti diversi del progetto convivevano quattro convenzioni su quale classe significasse "scarta il pacchetto":

Convenzione	Dove
la classe 6 è DROP	in un punto
la classe 5 è DROP	in un altro
$\text{DROP} = n_{\text{out}} - 1$	in un terzo
$\text{DROP} = \max(\text{etichette}) + 1$	dedotto dai dati

E sotto ce n'era una quinta, implicita e più insidiosa: che la classe k volesse dire "manda fuori dall'interfaccia k ". Funzionava per caso, perché nel modello depositato le porte sono numerate come le classi.

3.2 La soluzione: una catena esplicita

Adesso il significato è **dichiarato**, mai dedotto, e passa per una catena di quattro passi, ognuno con un responsabile diverso:



- **classe**: quello che il modello ha calcolato;
- **azione**: FORWARD, DROP o UNUSED — una classe mai addestrata esiste ed è dichiarata tale;
- **porta logica**: un numero che il *modello* conosce;
- **ifindex**: l'interfaccia vera, che solo il *nodo* conosce e che è diversa su ogni macchina.

Nel modello depositato la dichiarazione è:

classe	azione	porta logica
0-4	FORWARD	0-4
5	DROP	—
6	UNUSED	—

Il punto non è che adesso funziona: funzionava anche prima, per coincidenza numerica. Il punto è che adesso **smetterebbe di funzionare rumorosamente** se il modello cambiasse, invece di continuare a girare dando risposte plausibili e sbagliate.

3.3 Un errore che questa separazione ha fatto emergere

Il datapath teneva un contatore per classe, ma scriveva nella casella della *porta* invece che in quella della *classe*. Sembrava giusto perché le due numerazioni coincidono nel modello depositato.

Con un modello in cui non coincidono, il contatore avrebbe misurato la cosa sbagliata in silenzio. Separare i due spazi di indici ha reso l'errore visibile e correggibile.

Il motore non è lo scenario

4.1 Com'era

Il progetto era nato attorno a una topologia precisa: **Germany50**, una rete di 50 router usata come riferimento in letteratura, con due host attaccati, emulata con **Kathara** (container Docker connessi a formare la rete).

Due numeri erano ovunque nel codice:

$$n_{\text{nodi}} = 52, \quad n_{\text{interfacce}} = 6.$$

Il primo è $50 + 2$. Il secondo è il **grado massimo** della rete: il nodo con più vicini. Non è il grado di un nodo qualsiasi — tutti gli altri riempiono di zeri gli slot che non usano.

Questi numeri determinano la larghezza dell'ingresso:

$$n_{\text{in}} = \underbrace{6}_{\text{link}} + \underbrace{6}_{\text{iface}} + \underbrace{1}_{\text{ttl}} + \underbrace{52}_{\text{nodo}} = 65.$$

Il problema non era che esistessero, ma che il motore li conoscesse. C'era un *default* interno: se nessuno diceva niente, il motore assumeva 6 e 52. Ogni chiamante che dimenticava di specificare lo scenario risultava silenziosamente corretto per Germany50 e silenziosamente sbagliato altrove.

4.2 Com'è adesso: tre strati



La dipendenza punta **solo verso l'alto**. Il motore non sa che cosa sia Germany50.

Le dimensioni si risolvono in ordine: una variabile d'ambiente, poi un file di sistema, poi il blocco `trained_on` del descrittore del modello. Se nessuna delle tre risponde, il motore **si ferma** e dice dove ha cercato.

Un default comodo è il modo più efficace di nascondere un'assunzione. Preferire un errore esplicito a un valore plausibile è una scelta di progetto, non pignoleria: il valore plausibile non si manifesta mai come errore, solo come risultato sbagliato mesi dopo.

4.3 La prova

Non basta dirlo. Il motore genera oggi un datapath corretto per cinque topologie che non hanno niente a che vedere con quella d'origine:

iface	nodì	strati	n_{in}	n_{out}	vettore link
3	8	4-4	15	4	v[3]
4	16	6	25	5	v[4]
5	24	4-4-4	35	6	v[5]
2	6	3-3	11	3	v[2]
6	52	4-4	65	7	v[6]

Nessuna riga di C scritta a mano cambia fra una riga e l'altra.

Dall'emulatore al kernel vero

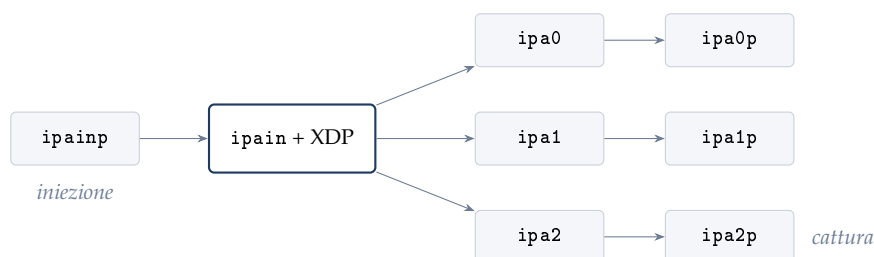
5.1 Che cosa provavano i test di prima

Fino a settembre ogni misura veniva da una chiamata di sistema, `BPF_PROG_TEST_RUN`: si dà al programma un buffer, lo si esegue N volte, e il kernel riporta quanti nanosecondi in media.

È utile e onesto per confrontare le tre pipeline fra loro. Ma non tocca un'interfaccia, non ha un driver, non consegna niente. Il programma poteva dire "redirigerei sulla porta 3" e nessuno verificava che un pacchetto ne uscisse davvero.

5.2 Il banco nuovo

Kathara è stato sostituito da qualcosa di più semplice e più vicino al vero: coppie di interfacce virtuali (veth) create sull'host.



Si inietta un frame da un capo, il programma XDP gira davvero, e si legge da quale interfaccia è uscito. Questo risponde alla domanda che `BPF_PROG_TEST_RUN` non può porre.

Due dettagli che sembrano minori e non lo sono

Modalità di attacco. XDP si può agganciare in due modi: *native*, dentro il driver, prima che esista la `sk_buff`; oppure *generic*, dopo. Il progetto usava sempre *generic*, e il motivo era scritto nel codice: serviva per le interfacce dentro i container dell'emulatore. Tolto l'emulatore, quel motivo cade. Adesso il default è *native*, la modalità *davvero in effetto* viene riletta dal kernel dopo l'aggancio, e un fallimento è un errore, non una retrocessione silenziosa.

Il vincolo delle veth. Redirigere verso una veth funziona solo se il lato ricevente ha a sua volta un programma XDP agganciato: il kernel alloca le code necessarie solo in quel caso. Senza, il redirect fallisce in silenzio e il pacchetto sparisce — indistinguibile da un modello che decide di scartare tutto. Il banco aggancia quindi un programma minimo a ogni capo.

5.3 Tre difetti del test, trovati misurando

Il banco nuovo ha trovato più difetti nei *test* che nel prodotto. Vale la pena elencarli, perché il motivo per cui erano invisibili è sempre lo stesso.

1. Il TTL non muove questo modello

La prima versione del test variava il TTL da 2 a 8 e riportava *"7 verifiche su 7 superate"*. Misurando lo spazio raggiungibile si scopre che con tutti i link attivi il modello risponde **sempre classe 2**, per qualsiasi TTL. Quel 7 su 7 era **un caso solo, ripetuto sette volte**.

È `link_state` la feature che sposta la decisione. Il test adesso *cerca* un ingresso per ogni classe, invece di spazzare alla cieca.

2. Un pacchetto del kernel contato come risultato

Su una veth appena creata il kernel manda del traffico suo (IPv6 multicast). Una volta è stato catturato e contato come la risposta della pipeline. È passato per caso, perché era sulla porta giusta.

Adesso la cattura filtra sui campi che il datapath non riscrive, e IPv6 viene disabilitato sulle interfacce del banco.

3. La porta 0 era invisibile per costruzione

L'ingresso era anche la porta logica 0. Una classe che inoltrava su quella porta rimandava il frame fuori dall'interfaccia da cui era entrato — dove la cattura non lo distingue da quello appena iniettato. Una decisione *corretta* sulla porta 0 non era osservabile.

Il banco ha adesso un ingresso dedicato, separato da tutte le porte.

Un test che non può fallire non è un test. In tutti e tre i casi il risultato era verde e privo di contenuto, e in tutti e tre il difetto è emerso solo mettendo in mezzo un kernel vero.

5.4 Il risultato

Tutte e tre le pipeline superano oggi la verifica end-to-end su un datapath reale: XDP native, redirect vero, pacchetto ripreso sulla porta d'uscita, e scarto verificato dal *contatore* e non dal silenzio — perché "non è arrivato niente" è anche ciò che si osserva quando il redirect si perde.

La feature che non c'era

6.1 Il sintomo

Il modello ha quattro ingressi. Uno di questi è *da quale interfaccia è arrivato il pacchetto*, codificato *one-hot*: un vettore di sei posizioni con un solo uno.

Il datapath leggeva l'indice dell'interfaccia dato dal kernel — l' *ifindex* — e lo usava direttamente come posizione nel vettore, con un controllo di validità:

$$1 \leq \text{ifindex} \leq n_{\text{interfacce}}.$$

Gli *ifindex* però li assegna il kernel, in ordine di creazione. Su un container appena avviato valgono 2, 3, 4. Su una macchina che ha creato qualche interfaccia virtuale valgono 205, 217, 229.

Il controllo falliva sempre. **Nessun bit veniva acceso.** Il modello girava di fatto su tre ingressi invece di quattro.



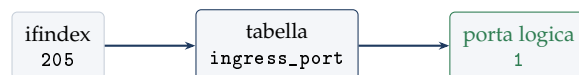
6.2 Perché nessun test lo vedeva

Perché nessun test controllava che quella feature *contribuisse* qualcosa. Riferimento e datapath la ignoravano entrambi, quindi **concordavano** — e l'accordo veniva registrato come successo.

Una pipeline aveva in effetti una tabella di traduzione, ma compilata dentro il programma come [2, 3, 4, 5, 6, 7]: la stessa assunzione, scritta in un altro modo. Le altre due usavano l'indice grezzo. Le tre pipeline sceglievano quindi *colonne diverse* per lo stesso pacchetto — una discrepanza documentata in un commento e mai risolta.

6.3 La correzione

La traduzione è diventata una **tabella a runtime**, riempita dal piano di controllo leggendo le interfacce del nodo:



È il **simmetrico** della tabella già usata in uscita: là "quale interfaccia realizza la porta che il modello ha scelto", qui "quale porta è l'interfaccia da cui il pacchetto è arrivato". Entrambe sono fatti del nodo, entrambe si risolvono quando il nodo esiste.

6.4 La prova che adesso è viva

Prima e dopo la correzione, gli ingressi che raggiungono ciascuna classe **cambiano**:

classe	prima (feature morta)	dopo (feature viva)
0	100110, ttl 27	100111, ttl 20
1	010000, ttl 2	010000, ttl 14
5	000000, ttl 2	000000, ttl 12

Stesso stato dei link, TTL diverso per ottenere la stessa classe: l'ingresso in più contribuisce. E il TTL ora sposta la decisione, cosa che prima non faceva.

6.5 Il prezzo, misurato

Una lettura di tabella in più per pacchetto:

	prima	dopo
letture per pacchetto (P1)	4,0	5,0
letture per pacchetto (P3)	26,0	28,0
istruzioni (P1)	997	971

Le istruzioni di P1 *calano*: la tabella ha sostituito un blocco switch a sei casi, e una lettura costa meno di un salto.

I numeri, e che cosa dicono davvero

7.1 La tabella

Misure su una macchina virtuale a 4 processori, modello 65-4-4-7, scala 24.

	baseline	P1	P2	P3
istruzioni caricate	155	971	16 185	11 826
salti fra programmi	0	1	1	3
letture di tabella	3,0	5,0	n.d.	28,0
latenza, esecuzione 1	35 ns	62 ns	243 ns	478 ns
latenza, esecuzione 2	28 ns	67 ns	247 ns	422 ns

Il *baseline* è un programma che analizza il pacchetto e lo rimanda fuori senza alcuna inferenza: è il pavimento contro cui si legge tutto il resto.

7.2 Primo risultato: la dimensione non predice la velocità

P3 ha il 27% di istruzioni in meno di P2 ed è quasi il doppio più lento.

Il motivo sta nelle due righe centrali: tre salti fra programmi invece di uno, e ventotto letture di tabella per pacchetto.

Il conteggio delle istruzioni misura quanto è *grande* il programma caricato, non quanto *lavoro* fa per pacchetto. In questo regime a dominare sono le letture di memoria e i salti, non l'aritmetica.

Questo si vede solo perché le quattro colonne vengono dalla stessa esecuzione, sulla stessa macchina, nello stesso minuto.

7.3 Secondo risultato: quanto valgono le latenze assolute

Le due esecuzioni in tabella sono lo **stesso identico binario**, a minuti di distanza.

	baseline	P1	P2	P3
scarto fra le due esecuzioni	−20%	+8%	+2%	−12%

Le **istruzioni sono identiche** nelle due esecuzioni: sono deterministiche. Le latenze no.

Su questa macchina virtuale una differenza di latenza sotto il 20% fra due misure prese in momenti diversi **non significa nulla**. Vale il confronto *dentro* una singola esecuzione.

Ogni cifra assoluta va letta come ordine di grandezza su questa VM, mai come prestazione del sistema.

Questo ha reso necessario rifare la tabella dei risultati: la versione precedente presentava tre esecuzioni etichettate come ripetizioni, ma venivano in realtà da *stati diversi del codice*.

7.4 Terzo risultato: la genericità ha un costo di verifica

P2 serve qualsiasi modello che stia sotto certi tetti dichiarati a priori (numero massimo di neuroni, di ingressi, di feature). È il suo pregio: non ricompila mai.

Il conto però lo paga il verificatore del kernel, che percorre ogni cammino del corpo srotolato. Con i tetti larghi il kernel si arrende:

```
processed 1000001 insns (limit 1000000) ... total_states 13864
```

Da notare: il programma è lungo circa 14 900 istruzioni, **ampiamente dentro** il limite di dimensione. Quello che finisce è la *complessità di verifica*, che è un limite diverso e cresce col **prodotto** dei tetti:

$$\text{costo di verifica} \sim H_{\max} \times F_{\max} \times N_{\max}.$$

È la misura concreta di quanto un programma "generico" possa esserlo. La genericità di P2 non è gratis: si paga in stati del verificatore, e il tetto si tocca con numeri ragionevoli (8 neuroni, 4 feature, 128 ingressi).

7.5 Che cosa questi numeri non dicono

BPF_PROG_TEST_RUN esegue il programma in un ciclo sullo stesso buffer. Niente scheda di rete, niente driver, nessuna allocazione, nessuna pressione di cache da traffico vero.

Il valore in "milioni di pacchetti al secondo" è semplicemente $1/\text{latenza}$: un **picco teorico**, non un throughput retto. Per quello serve traffico vero, generato a ritmo, e la misura di quanti pacchetti si perdono.

Quel che resta

8.1 Fatto

- semantica delle classi dichiarata, mai dedotta; catena classe $\rightarrow$ azione $\rightarrow$ porta logica $\rightarrow$ interfaccia;
- motore indipendente dallo scenario, dimostrato su cinque topologie diverse;
- emulatore rimosso; verifica end-to-end su datapath reale con XDP native;
- errore del bias nell'ultimo strato corretto;
- feature dell'interfaccia d'ingresso resa viva in tutte e tre le pipeline;
- tabella dei risultati rifatta su uno stato del codice solo.

8.2 Aperto

- **Throughput reale:** generare traffico a ritmo e misurare pacchetti al secondo, perdite e carico di CPU. È la misura che manca se la metrica principale è la velocità.
- **Versione intermedia fra P1 e P2:** pesi scritti nel codice ma descrittore letto a runtime. Oggi il salto fra P1 e P2 confonde due cambiamenti insieme, e non si può attribuire il costo all'uno o all'altro.
- **Hardware vero:** la scheda di rete di questa macchina virtuale è emulata e non supporta XDP native. I numeri assoluti restano non citabili come prestazioni di sistema.

8.3 Il filo conduttore

Guardando indietro, quasi tutti i problemi trovati a settembre hanno la stessa forma. Non erano calcoli sbagliati: erano **assunzioni non dichiarate** che risultavano vere per caso nell'unica configurazione mai provata.

Assunzione	Vera solo perché
classe k = interfaccia k	le porte erano numerate come le classi
DROP = ultima classe	nel modello depositato capitava così
$n_{\text{nod}} = 52$ di default	si provava una topologia sola
eth0 = ifindex 2	si provava dentro un container pulito
bias dell'uscita $\times s^0$	si provava coi link tutti attivi

E quasi tutte sono emerse allo stesso modo: cambiando *una* condizione che nessun test aveva mai cambiato.

Un test verde dice che il codice si comporta come il test si aspetta. Non dice che il test stia chiedendo qualcosa.